LAPORAN PRAKTIKUM

- **Identitas Praktikum**

  Nama MK : Praktikum Pemrograman Berorientasi Objek
  Kode MK : CAK2KAB4
  Bobot SKS : 3 SKS
  Tempat : L-MM
  Hari, tanggal : Jumat, ... April 2026
  Jam : 12:30-15:30
  Topik praktikum : Modul-2

- **Identitas Mahasiswa**

  Nama lengkap : Harald Riandi Rantetana Purukan
  NIM : 103112400130
  Program Studi : S-1 Teknik Informatika

- **Algoritma / studi kasus (jika ada) & Penjelasan**

  Pemodelan sistem informasi akademik menggunakan paradigma Pemrograman Berorientasi Objek (PBO) yang merepresentasikan hierarki dan asosiasi memori antara Fakultas, Program Studi (Prodi), dan Dosen. Sistem membuktikan perbedaan implementasi antara *Composition* (ketergantungan eksistensial absolut) dan *Aggregation* (ketergantungan referensial longgar).

Algoritma Eksekusi:

1. **Alokasi Memori Entitas Independen:** Tiga instans objek Dosen diciptakan. Objek-objek ini eksis secara independen di memori tanpa perlu terikat pada struktur apapun.
2. **Inisialisasi Entitas Induk (Root):** Instans Fakultas diinisialisasi sebagai *container* tertinggi dalam hierarki.
3. **Eksekusi Relasi Komposisi:** Metode tambahProdi dipanggil. Penciptaan objek Prodi dieksekusi secara internal oleh dan di dalam lingkup memori Fakultas. Ini mengunci siklus hidup Prodi; jika instans Fakultas dihapus dari memori, instans Prodi ikut musnah.
4. **Eksekusi Relasi Agregasi:** Alamat referensi dari objek Dosen diinjeksikan ke dalam struktur ArrayList milik Prodi via metode tambahDosen(). Ini membuktikan agregasi: Prodi mengumpulkan objek eksternal. Secara logis dibuktikan ketika

entitas dosen1 dipetakan ke prodiTI sekaligus prodiSI tanpa memerlukan duplikasi objek.

5. **Traversal Hierarkis:** Metode tampilkanData() memicu iterasi bersarang. Sistem membaca status internal Fakultas, mengiterasi daftar Prodi di dalamnya, dan setiap Prodi mengiterasi daftar referensi Dosen yang dipegangnya, menghasilkan output terminal seperti pada tangkapan layar.

- **Kode Program & Penjelasan**

Dosen.java

```
public class Dosen {
    private String namaDosen;
    private String nip;

    public Dosen(String namaDosen, String nip) {
        this.namaDosen = namaDosen;
        this.nip = nip;
    }

    public void mengajar() {
            System.out.println("Dosen " + this.namaDosen + " sedang
mengajar.");
    }

    public String getNama() {
        return this.namaDosen;
    }

    public String getNIP() {
        return this.nip;
    }
}
```

Penjelasan :

Deklarasi *blueprint* dasar untuk entitas Dosen. Enkapsulasi diterapkan dengan ketat melalui *modifier* private pada atribut, di mana akses data hanya dimungkinkan melalui *getter* (getNama(), getNIP()).

Fakultas.java

```
import java.util.ArrayList;

public class Fakultas {
    private String namaFakultas;
    private ArrayList<Prodi> daftarProdi;

    public Fakultas(String namaFakultas) {
        this.namaFakultas = namaFakultas;
        this.daftarProdi = new ArrayList<>();
    }

    public void tambahProdi(String namaProdi) {
        Prodi prodiBaru = new Prodi(namaProdi);
        this.daftarProdi.add(prodiBaru);
    }

    public Prodi getProdi(int index) {
        return this.daftarProdi.get(index);
    }

    public void tampilkanData() {
        System.out.println("Fakultas: " + this.namaFakultas);
        for (Prodi p : daftarProdi) {
            p.tampilkanData();
        }
    }
}
```

Penjelasan:

Implementasi relasi Komposisi *strict*. Metode tambahProdi(String namaProdi) menyembunyikan logika pembuatan objek Prodi (new Prodi(...)) dari dunia luar. Class pemanggil tidak memiliki akses untuk menyuntikkan objek Prodi yang sudah jadi. Keterikatan ini mutlak.

Prodi.java

```
import java.util.ArrayList;

public class Prodi {
    private String namaProdi;
    private ArrayList<Dosen> daftarDosen;

    public Prodi(String namaProdi) {
        this.namaProdi = namaProdi;
        this.daftarDosen = new ArrayList<>();
    }

    public void tambahDosen(Dosen dosen) {
        this.daftarDosen.add(dosen);
    }

    public void tampilkanData() {
        System.out.println("  Program Studi: " + this.namaProdi);
        System.out.println("  Daftar Dosen Pengajar:");
        if (daftarDosen.isEmpty()) {
            System.out.println("    - (Belum ada dosen)");
        } else {
            for (Dosen d : daftarDosen) {
                System.out.println("    - " + d.getNama());
            }
        }
    }
}
```

Penjelasan:

Implementasi relasi Agregasi. Atribut ArrayList<Dosen> berfungsi sebagai penyimpan koleksi *pointer* ke objek Dosen. Konstruktor tidak memaksa pembuatan objek Dosen baru, memvalidasi aturan sistem bahwa "Dosen tetap ada walaupun tidak terdaftar di Prodi".

Main.java

```
public class Main {
    public static void main(String[] args) {

        Dosen dosen1 = new Dosen("Dr. Alan Turing", "101010");
        Dosen dosen2 = new Dosen("Dr. Ada Lovelace", "010101");
        Dosen dosen3 = new Dosen("Dr. John von Neumann", "111111");

        Fakultas fasilkom = new Fakultas("Ilmu Komputer");

        fasilkom.tambahProdi("Teknik Informatika");
        fasilkom.tambahProdi("Sistem Informasi");

        Prodi prodiTI = fasilkom.getProdi(0); // Mendapatkan instans Teknik
Informatika
        Prodi prodiSI = fasilkom.getProdi(1); // Mendapatkan instans Sistem
Informasi

        prodiTI.tambahDosen(dosen1);
        prodiTI.tambahDosen(dosen2);
        prodiSI.tambahDosen(dosen3);

        prodiSI.tambahDosen(dosen1);

        fasilkom.tampilkanData();

        System.out.println("\n--- Eksekusi Method Dosen ---");
        dosen1.mengajar();
        dosen2.mengajar();
    }
}
```

Penjelasan:

Entry point eksekusi (*driver class*). Logika prosedural mengatur *state* dan interaksi antar objek, membuktikan bahwa seluruh *constraint* PBO (enkapsulasi, komposisi, agregasi) berfungsi.

- **Hasil Running Program**

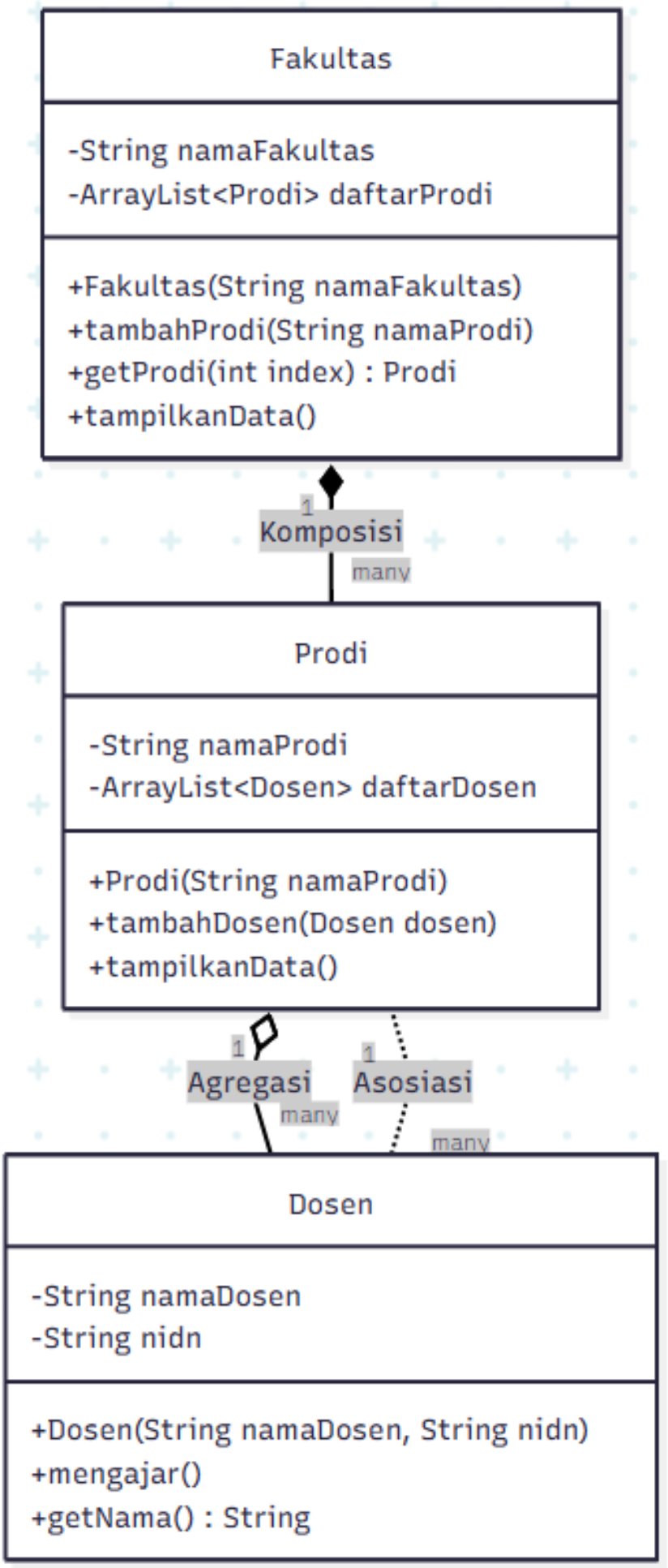
```
Fakultas: Ilmu Komputer
  Program Studi: Teknik Informatika
  Daftar Dosen Pengajar:
    - Dr. Alan Turing
    - Dr. Ada Lovelace
  Program Studi: Sistem Informasi
  Daftar Dosen Pengajar:
    - Dr. John von Neumann
    - Dr. Alan Turing

--- Eksekusi Method Dosen ---
Dosen Dr. Alan Turing sedang mengajar.
Dosen Dr. Ada Lovelace sedang mengajar.
PS D:\Documents\Semester 4\PBO\Praktikum\Modul 5>
```

Bagian A

1. Class Dosen
   - Atribut: String namaDosen, String nidn
   - Method: Dosen(String, String), mengajar(), getNama()
2. Class Prodi
   - Atribut: String namaProdi, ArrayList<Dosen> daftarDosen
   - Method: Prodi(String), tambahDosen(Dosen), tampilkanData()
   - Relasi: Agregasi (Prodi -> Dosen). Prodi mengumpulkan referensi objek Dosen yang dibuat di luar dirinya.
3. Class Fakultas
   - Atribut: String namaFakultas, ArrayList<Prodi> daftarProdi
   - Method: Fakultas(String), tambahProdi(String), getProdi(int), tampilkanData()
   - Relasi: Komposisi (Fakultas -> Prodi). Objek Prodi diinstansiasi secara eksklusif di dalam method Fakultas, memastikan keterikatan mutlak.

Diagram Class UML:



Bagian B

Sudah di atas